Database Function Questions

```
def  retrieve_customer_details(address,conn):
    #WRITE YOUR CODE HERE FOR IMPLEMENTING THE
FUNCTION
    cursor = conn.cursor()

    query = """
        SELECT room_no, cust_name, address, phone_number,
from_date, to_date
        FROM hotel
        WHERE address = :1
        AND (to_date - from_date)>3
    """
    cursor.execute(query,(address,))
    records = cursor.fetchall()

    if len(records) == 0:
        return []
    hotel_list = []

    for row in records:
        hotel_obj = hl.Hotel(
                row[0], row[1], row[2],
                row[3], row[4], row[5]
            )
        hotel_list.append(hotel_obj)



    return hotel_list

 #### WRITE YOUR CODE HERE FOR GETTING USER
INPUTS, FUNCTION CALL AND OUTPUT STATEMENTS
address = input("Enter the Address: ")

result = retrieve_customer_details(address,conn)
if not result:
    print("Invalid Address")
else:
    count=1
    for hotel in result:
```

```
        print("Customer details", count)
        print("Room no:", hotel.get_room_no())
        print("Customer Name:", hotel.get_cust_name())
        print("Phone Number:", hotel.get_phone_number())
        no_of_days =  (hotel.get_to_date()-
hotel.get_from_date()).days
        print("No of days:", no_of_days)
        count+=1



def retrieve_flight_by_id(flight_id,conn):
    cursor = conn.cursor()
    select = """SELECT * FROM flight WHERE flightId =
:param1"""
    row = cursor.execute(select,param1=flight_id)
    row = cursor.fetchone()
    if row!= None:
        flight_obj = fl.Flight(row[0],row[1],row[2],row[3],row[4])
        return flight_obj
    else:
        return row




 #### WRITE YOUR CODE HERE FOR GETTING USER
INPUTS, FUNCTION CALL AND OUTPUT STATEMENTS
flight_id = int(input("Enter the flight id:"))

flight_obj = retrieve_flight_by_id(flight_id,conn)

if flight_obj == None:
    print("Invalid Flight ID")
else:
    print("Flight ID:", flight_obj.get_flight_id())
    print("Source:", flight_obj.get_source())
    print("Destination:", flight_obj.get_destination())
    print("No of seats:", flight_obj.get_no_of_seats())
    print("Flight Fare:", flight_obj.get_flight_fare())
```

I HAVE DOUBT

first, why are we passing conn while calling function when conn is declared and define global
second, when excute happen in one execute we use placeholder=value when we have :param1
but when we used :1 then in execute we passed tuple with param so which oen to used what the difference
third=in one execute we have taking var and in one we dont have so do we need it or not
also what does fetchone/fetchmany/fetchall return

Good questions 👍 These are common doubts when working with **Python DB API (like cx_Oracle / sqlite3 / psycopg2 etc.)**. I'll explain each clearly.

1️⃣ Why pass `conn` to the function if it is global?

Example:

```python
def retrieve_customer_details(address, conn):
```

Even if `conn` exists globally, **passing it as a parameter is good practice**.

❌ Using global connection

```python
conn = connect()

def retrieve_customer_details(address):
    cursor = conn.cursor()   # using global variable
```

Problems:

- Function depends on **global state**
- Harder to test
- Harder to reuse

✅ Passing connection

```python
```

```
def retrieve_customer_details(address, conn):
    cursor = conn.cursor()
```

Advantages:

✔️ Function becomes **independent**
✔️ Can use **different connections**
✔️ Easier **testing and debugging**

Example:

```python
conn1 = connect_db1()
conn2 = connect_db2()

retrieve_customer_details("Delhi", conn1)
retrieve_customer_details("Mumbai", conn2)
```

So even if `conn` is global, **passing it is better design**.

2️⃣ `:param1` vs `:1` placeholders

This depends on the **type of binding** used.

There are **two styles**.

Style 1 — Named parameters

Query:

```python
SELECT * FROM flight WHERE flightId = :param1
```

Execute:

```python
cursor.execute(select, param1=flight_id)
```

Here:

```
:param1  → named placeholder
```

You pass **name=value**

✔️ More readable

Style 2 — Positional parameters

Query:

```python
SELECT * FROM hotel WHERE address = :1
```

Execute:

```python
cursor.execute(query, (address,))
```

Here:

```
:1 → first parameter
:2 → second parameter
```

Example:

```python
query = """
SELECT * FROM hotel
WHERE address = :1 AND room_no = :2
"""

cursor.execute(query, ("Delhi",101))
```

📌 Difference

Type	Query	Execute
Named	`:param1`	`param1=value`
Positional	`:1`	`(value,)` tuple

⭐ Recommendation

Use **named parameters** when possible because:

✔️ easier to read
✔️ easier for multiple parameters

3️⃣ Why `cursor.execute(select,param1=flight_id)` sometimes has variable and sometimes not?

`execute()` always has **two parts**

```
cursor.execute(SQL_QUERY , parameters)
```

But if **no parameters**, then:

```python
cursor.execute("SELECT * FROM flight")
```

If **parameters exist**, then:

```python
cursor.execute(query,(address,))
```

or

```python
cursor.execute(query,param1=value)
```

So parameters are **only required when placeholders exist**.

4️⃣ What do `fetchone()`, `fetchmany()`, `fetchall()` return?

These methods **retrieve query results**.

`fetchone()`

Returns **one row at a time**

Example result:

```python
row = cursor.fetchone()
```

Return value:

```
(row1_col1, row1_col2, row1_col3)
```

Example:

```
(101,'Chetan','Delhi')
```

If no rows:

```
None
```

`fetchall()`

Returns **all rows**

```python
rows = cursor.fetchall()
```

Result:

```
[
 (101,'Chetan','Delhi'),
 (102,'Rahul','Mumbai'),
 (103,'Priya','Pune')
]
```

Type:

```
list of tuples
```

`fetchmany(size)`

Returns **specified number of rows**

```python
rows = cursor.fetchmany(2)
```

Result:

```
[
 (101,'Chetan','Delhi'),
 (102,'Rahul','Mumbai')
]
```

Quick Visual Summary

Method	Returns
`fetchone()`	One tuple
`fetchmany(n)`	List of n tuples
`fetchall()`	List of all tuples

Example

Table:

id	name
1	A
2	B
3	C

`fetchone()`

```
(1,'A')
```

`fetchmany(2)`

```
[(1,'A'),(2,'B')]
```

`fetchall()`

```
[(1,'A'),(2,'B'),(3,'C')]
```

🔥 One small bug in your code

Here:

```python
row = cursor.execute(select,param1=flight_id)
row = cursor.fetchone()
```

`execute()` **does not return rows**.

Correct:

```python
cursor.execute(select,param1=flight_id)
row = cursor.fetchone()
```

⭐ Interview Tip

Remember:

```
execute() → runs query
fetch*() → retrieves result
```

If you want, I can also explain **5 very common mistakes students make in Python database programs** (these questions often come in practical exams).

SELECT * FROM flight WHERE flightId = :param1
if we have to work with two param then hwo woulld be do that with query and execute method hwo would eedefine it

Why cursor.execute(select,param1=flight_id) sometimes has variable and sometimes not?
you still dont tell me do e need or not the return storing variable do we need that or not

Let's go step-by-step.

1️⃣ If we have two parameters with `:param1`

You simply create **two named placeholders** in the query.

Query

```python
query = """
SELECT *
FROM flight
WHERE source = :param1
AND destination = :param2
"""
```

Execute

```python
cursor.execute(query, param1=source, param2=destination)
```

Example:

```python
source = "Delhi"
destination = "Mumbai"

cursor.execute(query, param1=source, param2=destination)
```

Another example

```python
query = """
SELECT * FROM flight
WHERE flightId = :param1
AND source = :param2
"""
```

Execute:

```python
cursor.execute(query, param1=flight_id, param2=source)
```

2️⃣ Same query using positional placeholders

Instead of names you can use numbers.

Query

```python
query = """
SELECT * FROM flight
WHERE source = :1
AND destination = :2
"""
```

Execute

```python
cursor.execute(query, (source, destination))
```

Notice:

```python
(source, destination)
```

is a **tuple**.

Summary

Placeholder Type	Query	Execute
Named	`:param1 :param2`	`param1=value1, param2=value2`
Positional	`:1 :2`	`(value1,value2)`

3️⃣ Your second doubt (very important)

You asked:

> Why sometimes we write
> `row = cursor.execute(...)`
> Do we need to store it or not?

The correct answer:

❌ **You do NOT need to store `execute()` result.**

Because `execute()` **returns None** in most DB APIs.

Correct way:

```python
cursor.execute(query, param1=flight_id)
row = cursor.fetchone()
```

Wrong / unnecessary

```python
row = cursor.execute(query, param1=flight_id)
```

`row` will just store:

```
None
```

So it is useless.

4️⃣ Correct pattern you should always use

Step 1: Execute query

```python
cursor.execute(query, param1=flight_id)
```

Step 2: Fetch data

```python
row = cursor.fetchone()
```

or

```python
rows = cursor.fetchall()
```

5️⃣ Complete example (2 parameters)

```python
def retrieve_flight(source, destination, conn):

    cursor = conn.cursor()

    query = """
    SELECT * FROM flight
    WHERE source = :param1
    AND destination = :param2
    """

    cursor.execute(query, param1=source, param2=destination)

    rows = cursor.fetchall()

    return rows
```

6️⃣ Visual flow of SQL execution

```
cursor.execute()  → run query
        ↓
```

```
fetchone()      → one row
fetchmany()     → few rows
fetchall()      → all rows
```

7️⃣ One more small improvement for your code

Instead of:

```python
if row != None:
```

Better:

```python
if row is not None:
```

This is **Python standard style**.

✅ If you want, I can also explain **the 6 most common mistakes students make in Python database practical exams** (very useful if this is for viva/practical).

```
cursor = conn.cursor()

        query = f"""
            SELECT train_number, train_name, source,
destination,
                ac1, ac2, ac3, sleeper, seater
            FROM train
            WHERE LOWER(source) = :source
              AND LOWER(destination) = :destination
              AND {coach_column} > 0
            ORDER BY train_number
        """

        cursor.execute(
            query,
            {
```

```
        'source': source.lower(),
        'destination': destination.lower()
    }
)
```

here we have used dict cant we just used normal way like you said why to used dict

Good observation. Yes 👍 you **can use the normal way**, but in this case a **dictionary is used because the placeholders are named** (`:source`, `:destination`). Let's break it clearly.

1️⃣ Your query uses named placeholders

```python
WHERE LOWER(source) = :source
AND LOWER(destination) = :destination
```

Here the placeholders are:

```
:source
:destination
```

So Python needs **values mapped to those names**.

That is why this is used:

```python
cursor.execute(
    query,
    {
        'source': source.lower(),
        'destination': destination.lower()
    }
)
```

This dictionary means:

```
:source      → source.lower()
:destination → destination.lower()
```

2️⃣ Why dictionary is commonly used

Because it clearly maps parameters.

Example:

```python
{
 'source': 'delhi',
 'destination': 'mumbai'
}
```

So the database knows exactly where to substitute values.

3️⃣ Can we use the normal way?

Yes, but it depends on the **placeholder style**.

Option 1 — Named parameters (dictionary)

Query:

```python
WHERE source = :source AND destination = :destination
```

Execute:

```python
cursor.execute(query,{
    "source":source,
    "destination":destination
})
```

Option 2 — Positional parameters (tuple)

Query:

```python
WHERE source = :1 AND destination = :2
```

Execute:

```python
cursor.execute(query,(source,destination))
```

4️⃣ Why dictionary is preferred sometimes

Imagine many parameters.

Example:

```sql
WHERE source=:source
AND destination=:destination
AND train_number=:train
AND coach=:coach
```

Using dictionary:

```python
cursor.execute(query,{
    "source":source,
    "destination":destination,
    "train":train,
    "coach":coach
})
```

This is **very readable**.

5️⃣ Important: Why not use tuple here?

Because your query contains:

```
:source
:destination
```

These are **named placeholders**, not positional ones.

If you try:

```python
cursor.execute(query,(source,destination))
```

it **may fail depending on the driver**.

6️⃣ Quick comparison

Query Placeholder	Execute Parameter
`:1 :2`	`(value1,value2)`
`:name`	`{'name':value}`

7️⃣ One more important thing in your code

This line:

```python
AND {coach_column} > 0
```

is **string interpolation using f-string**.

It dynamically inserts the column name.

Example result:

```sql
AND ac2 > 0
```

This **cannot be passed as SQL parameter**, so f-string is needed here.

✅ Final takeaway

Use:

- **Dictionary** → when placeholders are **named**
- **Tuple** → when placeholders are **positional**

If you want, I can also show you **the safest way to write dynamic SQL like** `{coach_column}` **without risking SQL injection**, which is a very important concept in database programming.

can we use :param1 and :param2 when pass like normal dont we do that

Yes 👍 you **can use** `:param1` **and** `:param2` **and pass them in the normal way**, but you must understand **what "normal way" means depending on the database driver**.

Let's clarify.

1️⃣ Using `:param1` and `:param2` with named parameters

Query:

```python
query = """
SELECT * FROM flight
WHERE source = :param1
AND destination = :param2
"""
```

Method 1 — Dictionary (most common)

```python
cursor.execute(query, {
    "param1": source,
    "param2": destination
})
```

Here:

```
:param1 → source
:param2 → destination
```

This is the **most reliable method**.

2️⃣ Using `:param1` with keyword arguments

Some drivers (like **cx_Oracle**) allow this:

```python
cursor.execute(query, param1=source, param2=destination)
```

But **not all Python DB drivers support this**, so dictionary is safer.

3️⃣ Using tuple `(value1, value2)`

This **only works with positional placeholders**.

Example:

Query

```python
query = """
SELECT * FROM flight
WHERE source = :1
AND destination = :2
"""
```

Execute

```python
cursor.execute(query, (source, destination))
```

Here:

```
:1 → source
:2 → destination
```

4️⃣ Why tuple does NOT match `:param1`

Tuple parameters are **position based**, not name based.

So this:

```python
cursor.execute(query,(source,destination))
```

expects placeholders like:

```
:1
:2
```

not:

```
:param1
:param2
```

5️⃣ Quick rule (very important)

Placeholder	Execute Parameter
`:1 :2`	`(value1,value2)`
`:name`	`{'name':value}`
`:name`	`name=value` (sometimes supported)

6️⃣ Example combining everything

Named style

```python
query = """
SELECT * FROM flight
WHERE source = :src
AND destination = :dest
"""

cursor.execute(query,{
    "src":source,
    "dest":destination
})
```

Positional style

```python
```

```
query = """
SELECT * FROM flight
WHERE source = :1
AND destination = :2
"""

cursor.execute(query,(source,destination))
```

⭐ Recommendation (industry practice)

Use **named parameters with dictionary**:

```python
cursor.execute(query,{
    "source":source,
    "destination":destination
})
```

because it is:

✔️ clearer
✔️ safer
✔️ easier to maintain

✅ If you want, I can also explain **one tricky exam question about SQL placeholders in Python that confuses 90% of students**, and how to answer it in interviews or viva.